

Inheritance is one of the fundamental design principles in OOP. BUT relational databases don't support inheritance

JPA suggests different strategies to support inheritance hierarchies.

Hibernate Inheritance strategies

1 Single Table Strategy (default inheritance strategy)

In this inheritance, a single table is used to store all the instances of the entire inheritance hierarchy. The Table will have a column for every attribute of every class in the hierarchy. Discriminator columns identifies which class a particular row belongs.

Annotations used in super class

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "emp_type")
@Table(name = "employees")
public class Employee {...}
```

In sub class :

```
@Entity
@DiscriminatorValue("worker")
public class Worker extends Employee {...}
```

```
@Entity
@DiscriminatorValue("mgr")
public class Manager extends Employee {...}
```

With this mapping strategy, only a single table will be created for both concrete classes (Manager and Worker). Hibernate will create a discriminator column named emp_type to differentiate each concrete type. The value of this column will be either worker or mgr

Use case : In case of larger no of common fields between sub classes.

2. Joined Table Strategy

This is the most logical solution, as it mirrors the object structure in the database. In this approach, a separate database table is defined for each of the class in the hierarchy and each table stores only its local attributes.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
}
```

```
@Entity
public class Book extends Product {
    private String isbn;
}
```

The above mapping will create two tables, one for super class Product and another for entity Book. Both the tables will have Product id column. The primary key of table Book will have foreign key relationship with primary key of Product table.

3. Table Per class

Not supported by all JPA vendors .

A separate table is defined for each concrete class in the inheritance hierarchy to store all the attribute of that class and all its super classes.

4. @MappedSuperClass

We want to extract common behavior in a super class in JPA entities but without having a table for that super class. How would you achieve that?

If we create a normal class as the super class, then as per JPA specifications, the fields for that class are not persisted in the database tables. We need to create a super class extracting the common fields and then annotate that class with @MappedSuperClass in order to persist the fields of that super class in subclass tables. A mapped super class has no separate table defined for it.

MappedSuperClass example

@MappedSuperclass

```
public class Person {
```

```
    @Id
```

```
    private Long id;
```

```
    private String name;
```

```
}
```

@Entity

```
public class Employee extends Person {
```

```
    private String company;
```

```
}
```

The above configuration will result in a single table for employee with 3 columns (id, name and company), 1 from Employee class itself and 2 are inherited from Person class. Person class will never have its own table in this case.